Ready Scout

Disaster Management Platform

Project study guide and interview preparation playbook

React + Vite | Node.js + Express | MongoDB Atlas | Flask + Gemini

Repository-grounded edition | Snapshot: 24 October 2026

A practical guide to understanding the product, tracing its code, explaining its design, and discussing its limits with confidence.

Accuracy note: This guide describes the code and configuration currently present in the repository. It distinguishes implemented features from scaffolds, third-party dependencies, and deployment settings that still require valid environment credentials.

How to use this guide

Work through the phases in order for a full interview-preparation path, or jump to the section you need. The last two phases provide practice answers and a short review sheet.

Phase	What you will learn
1. Master overview	What Ready Scout does, who it serves, its workflows and architecture.
2. Codebase deep dive	How React, Express, MongoDB, Flask, alerts, the locator and drill game connect.
3. Technologies and design	Why the stack fits the project and where service boundaries sit.
4. Interview question bank	Project, React, API, database, authentication, and integration questions.
5. Technology study roadmap	A focused plan for strengthening project-related concepts.
6. System design and scalability	Request flows, scaling choices, caching, deployment and reliability.
7. Security and failure analysis	Current risks, failure modes, mitigations and honest limitations.
8. Mock interview	A sample interview narrative with follow-up prompts.
9. Final interview cheat sheet	Short pitch, architecture recap, key terms and claims to avoid.

Study method: For each feature, explain the user problem, the request path, the data shape, the failure case, and one improvement. If an item is not currently implemented, describe it as a proposed next step rather than a shipped capability.

Project snapshot: The main app is React/Vite with React Router. Express serves APIs and can serve the built frontend. The repository also includes a Flask/Gemini chatbot and a separate Vite quiz project. Nearby-place lookup currently calls Geoapify Places; the backend needs a valid server-side key for it to return places.

PHASE 1

Master project overview

Understand the platform as a learning and preparedness product, not as a public-safety dispatch system.

1. Project in plain English

Ready Scout is a disaster-preparedness learning platform. Students can explore preparedness content, alerts, quizzes, a leaderboard, and an earthquake drill simulation. A nearby-services page maps emergency resources. A separate chatbot answers disaster-management learning questions using a Flask service backed by Gemini.

2. Problem and intended users

The product centralizes preparedness learning and awareness features in one web application. Its primary audience is students; the codebase also contains a teacher-facing route and role field. The product provides educational material and links to public information. It does not replace official emergency alerts, local authorities, or emergency services.

3. Main user workflows

- Browse and learn: A visitor opens the Vite/React app, navigates through dashboard modules, and reads disaster-preparedness guidance.
- Account access: A user registers or logs in through Express. The backend uses MongoDB/Mongoose, bcrypt password hashing, and JWTs; Google OAuth is handled by Passport.js when configured.
- View alerts: The alert ticker loads the Express alerts endpoint. Alert data includes India NDMA SACHET/CAP information and client-side feed integrations such as USGS and GDACS, depending on the view/configuration.
- Find nearby services: The browser obtains the user location (or falls back to New Delhi), calls /api/nearby, and displays normalized places in a Leaflet map and sortable list.
- Practice earthquake actions: The interactive Phaser 3 game runs as a browser game from public assets. The separate quiz app is another Vite project.
- Ask the chatbot: The React chat UI sends a message to Flask; Flask adds a disaster-education system instruction and calls Gemini generateContent.

4. Architecture at a glance

Layer	Responsibility	Current technology
Web client	Pages, routing, state, user interactions, maps and charts	React 19, Vite, React Router, Tailwind CSS
Application API	Auth, profiles, alerts, nearby lookup, uploads, static build hosting	Node.js, Express 5
Persistence	User records and profile details	MongoDB Atlas through Mongoose
Chatbot service	Prompt shaping and model call	Flask, Python requests, Gemini API
Interactive learning	Earthquake simulation and standalone quiz	Phaser 3 game; separate React/Vite quiz
External data/services	Alerts, place search, map tiles	NDMA SACHET/CAP, USGS, GDACS, Geoapify, OpenStreetMap tiles

PHASE 2

Codebase deep dive

Follow an interaction from browser action to route, service, data source and response.

1. Repository reading order

Start here	Why
README.md	Local development, deployment configuration, project stack and required environment variables.
frontend/src/main.jsx and App.jsx	React bootstrapping and client-side route table.
frontend/src/app/context/AuthContext.jsx	Registration, login, logout, local user state and token refresh calls.
backend/index.js	Express middleware, service mounting, OAuth callbacks and static frontend serving.
backend/routes/authRoutes.js and userRoutes.js	Registration, login, token refresh, profile and account operations.
backend/models/User.js and middleware/authMiddleware.js	MongoDB schema, password hashing, JWT signing and route protection.
backend/routes/alertRoutes.js and nearbyRoutes.js	Alert transformations and Geoapify-to-frontend normalization.
flask/app.py	Flask chat endpoint, context instruction and Gemini request.
frontend/public/earthquake_drill/index.html and game.js	Standalone Phaser 3 game and gameplay lifecycle.

2. Startup and runtime boundaries

During local development, Vite runs at port 5173 and proxies /api and /uploads requests to Express on port 5000. The Flask chatbot starts separately at port 5001 by default. In production, Vercel serves the built Vite assets and rewrites API paths to the Render Express service, or Express can serve frontend/dist in a single-service arrangement. The standalone quiz has its own Vite server and package manifest.

3. Route map

Route	Purpose / behavior
POST /api/auth/register	Accepts multipart form data, optionally uploads a profile image, creates a user, and returns a signed JWT.
POST /api/auth/login	Validates email/password and returns user fields plus JWT.
GET /api/auth/google and callback	Starts Passport Google OAuth and redirects back to the frontend with token and user identifier.
POST /api/auth/refresh	Accepts a token and attempts to issue a renewed token for the associated account.
GET /api/users/profile	Protected profile read using Bearer JWT.
DELETE /api/users/delete-account	Protected account removal.
GET /api/alerts	Fetches and normalizes NDMA SACHET alert summaries; returns an empty collection on provider failure.
GET /api/alerts/cap/:identifier	Fetches CAP XML and parses human-readable alert details.
GET /api/nearby?lat=&lon;=&radius;=	Searches Geoapify Places and returns OSM-like normalized elements for the map/list.
POST /api/upload	Stores a profile image and returns its path.

4. Authentication walkthrough

Registration sends FormData so an optional profile picture can travel with user fields. The Express route checks required values, performs duplicate email/username queries, creates a Mongoose user, then signs a one-hour JWT. The User model hashes passwords in a pre-save hook with bcryptjs. Login retrieves the user by email, compares the password hash, and returns a JWT. Protected routes read the Bearer token, verify its signature and expiration, load the user, and attach it to req.user.

5. Nearby-places walkthrough

The nearby page requests browser geolocation. If permission is denied or unavailable, it defaults to New Delhi. It calls /api/nearby with a 5 km radius. The Express route validates coordinates, coalesces identical concurrent requests, uses a short in-memory cache and a stale-result window, then asks Geoapify for healthcare, police, fire-station and shelter/emergency categories. The backend converts GeoJSON features into a compact element shape; the React page classifies, computes Haversine distance, sorts, filters, and displays results in a list and Leaflet map.

Operational status to know: this repository expects GEOAPIFY_API_KEY on the backend. The current app will show an authentication error if the configured value is missing, malformed, restricted, or not accepted by Geoapify. Never place this key in VITE_* variables or commit its value.

6. Alert and chatbot flows

The Express alert route requests NDMA SACHET summaries, creates stable IDs, sorts by time and limits the response. A CAP detail route fetches XML and extracts language-specific description and instruction fields. Separate client utilities parse USGS GeoJSON and GDACS RSS feeds. The chatbot sends a user message to Flask; Flask supplies a fixed disaster-education system instruction, posts to Gemini, and returns extracted text. Provider errors need bounded timeouts and safe client-facing messages.

PHASE 3

Technologies and design decisions

Explain what each technology does and why the boundaries matter.

Technology	Role in Ready Scout	Trade-off / interview explanation
React	Reusable interactive UI components and context-based shared state.	Component composition helps split dashboards and widgets; state ownership and effect cleanup must be deliberate.
Vite	Frontend dev server, module handling and production bundling.	Fast feedback; production build is static files that need a host or backend server.
React Router	Client-side navigation between pages.	Direct route refresh needs SPA fallback/rewrite configuration.
Tailwind CSS	Utility-first styling for the main web app.	Fast layout work; shared tokens/components help avoid style drift.
Node.js + Express	HTTP API, middleware, auth, data access and optional static hosting.	Flexible and simple for a small REST API; validation, error contracts and service boundaries need discipline.
MongoDB Atlas + Mongoose	Cloud document database and object modeling.	Flexible user documents; indexes, schema validation and connection security still matter.
Passport.js + Google OAuth	OAuth 2.0 login strategy and callback processing.	Provider authentication complements app JWT auth; callback URL and session configuration must match each environment.
Flask + Gemini API	Small Python service that calls the generative model.	Separates AI provider logic; needs auth, timeouts, quotas, prompt controls and secret management.
Leaflet + OSM tiles	Interactive map rendering and attribution.	Map presentation and POI retrieval are distinct: Leaflet renders; Geoapify supplies place features.
Phaser 3	Canvas-based earthquake drill simulation.	Scene/state lifecycle and keyboard input suit game-like interactions.

Design boundary to emphasize

Leaflet is not the source of nearby-place data. It renders map tiles and markers. Geoapify Places currently supplies the points of interest; its dataset is based on OpenStreetMap. The frontend computes distance labels and display ordering. This separation means the map renderer can remain if the POI provider changes.

Data formats that cross boundaries

- Auth: JSON or multipart registration; JSON login response includes user properties and JWT.
- API auth: Authorization: Bearer <token>.
- Alerts: normalized JSON summaries; CAP details are parsed from XML into JSON.
- Nearby: GeoJSON features become normalized element-like records before reaching React.
- Chat: JSON request with a message; JSON response with model text or error.
- Game: browser-side Phaser scenes communicate through local game state, not the Express API unless persistence is separately implemented.

PHASE 4

Interview question bank

Practice concise answers, then expand with a request trace or design trade-off.

What is Ready Scout?

A disaster-preparedness education platform. It combines React learning pages, account-backed progress features, alerts, an emergency-services map, an earthquake simulation, and a disaster-focused chatbot. It is educational software, not an emergency dispatch system.

Why migrate the frontend from Next.js to Vite?

The project now uses a standalone React SPA with React Router and a separate Express API. Vite provides a direct development server and static production bundle without depending on Next.js server rendering or API routes. Express owns backend routes; Vercel can host the static app and rewrite API traffic.

What happens when Vite starts locally?

Vite serves the frontend on port 5173, and its proxy forwards /api and /uploads to Express on port 5000. The Flask chatbot runs separately on port 5001. MongoDB is reached by the backend through MONGO_URI.

How does password authentication work?

The API creates a Mongoose User, hashes the password with bcryptjs before save, checks the hash on login, signs a JWT, and expects protected routes to receive it as a Bearer token.

How does Google login fit with JWT?

Passport handles the Google OAuth redirect and callback. After authentication, Express signs the app JWT and redirects the browser back to the frontend. OAuth proves identity to the app; the JWT is the app credential used for API calls.

What does MongoDB Atlas store?

The current Mongoose model stores user identity, username, email, password hash, profile image path, role, Google ID, and timestamps. The Atlas connection is supplied through MONGO_URI.

How does the nearby map work?

The browser chooses a location, requests nearby places from Express, receives normalized places, sorts them by Haversine distance, and shows them in a list and Leaflet map. Geoapify is the current places provider; OpenStreetMap is the underlying source data and tile contributor attribution.

Why does nearby search need a backend?

The Geoapify key should remain server-side. The backend can validate coordinates, cache results, merge duplicate in-flight requests and normalize provider output so the frontend does not depend directly on the provider schema.

How is caching implemented for nearby results?

The Express route groups nearby requests into coordinate/radius cache keys, keeps fresh results briefly, shares concurrent requests, and may serve stale results for a bounded period if a provider request fails. This reduces repeated calls but is process-local, not shared across instances.

How does alert normalization work?

External alerts have different formats. The app maps provider fields to common properties such as event/title, time, location, severity and link. CAP XML details are parsed into readable JSON fields for the detail page.

What is CAP?

The Common Alerting Protocol is a structured alert format. Ready Scout’s backend retrieves CAP XML for a selected SACHET alert and parses headline, description, instruction, severity, timing and areas.

How does the chatbot work?

The React UI calls Flask. Flask wraps the message with a disaster-education system instruction, calls Gemini generateContent, extracts candidate text and returns JSON to the browser.

Why separate Flask from Express?

It isolates the Python/Gemini integration from the Node application API and allows the services to deploy independently. The cost is another runtime, URL, secret, health check and failure boundary.

How does the Phaser drill work?

A standalone page loads Phaser 3 and game.js. Scenes manage preload, gameplay and result states; keyboard input moves the character and duck action, while collision and timer logic determine outcomes.

How would you improve the project first?

Validate environment configuration at startup, add integration tests for auth/alerts/nearby/chat, make external calls bounded and observable, harden JWT refresh and OAuth token delivery, add upload limits, and make provider failures visible without exposing secrets.

What is a key limitation of the nearby feature?

Place completeness depends on the provider’s OpenStreetMap-derived coverage and the configured search categories. The current service also depends on a valid Geoapify API key and provider availability; results should not be presented as guaranteed emergency coverage.

Is the app production-ready for emergencies?

No. It is an educational prototype. It depends on external feeds, geolocation permission, map tile availability and configured credentials. Users should rely on official local authorities for emergency decisions.

How do you distinguish authentication and authorization?

Authentication establishes who the user is, such as password login or Google OAuth. Authorization checks what an authenticated user can do, such as a role-specific route or protected account operation.

What is the difference between Leaflet and Geoapify?

Leaflet is the frontend map-rendering library. Geoapify is the current nearby places search API. OpenStreetMap contributes the map tiles and place-data source used by the services, with attribution obligations.

What would you monitor?

API status and latency, MongoDB connection health, OAuth callback failures, provider status/latency and quota, chatbot errors and latency, cache hit rates, frontend route failures, and map/alert availability. Avoid logging tokens, passwords, prompt secrets or API keys.

PHASE 5

Technology study roadmap

A focused plan to strengthen the concepts most likely to come up in a project interview.

Order	Study topic	Be able to explain
1	HTTP and REST	Methods, status codes, request validation, JSON contracts, CORS, timeouts and idempotency.
2	React and Vite	Component state, context, useEffect cleanup, lazy loading, SPA routing, dev proxy and build output.
3	Node.js and Express	Middleware order, routers, async error handling, environment config, static serving and reverse proxies.
4	MongoDB and Mongoose	Documents, schema validation, indexes, unique constraints, connection pooling, Atlas access lists and backups.
5	JWT and OAuth 2.0	Signing vs decoding, expiration, refresh-token design, OAuth authorization-code flow, callback state and secure token storage.
6	External APIs	Provider contracts, rate limits, retries/backoff, caching, circuit breakers, attribution and graceful degradation.
7	Flask and Gemini	Flask routing, Python requests, prompt roles, timeouts, response parsing, safety, quotas and privacy.
8	Leaflet and Phaser	Map coordinates/markers/tiles vs POI data; Phaser scenes, game loop, input, collisions and lifecycle.
9	Deployment	Vercel rewrites, Render service variables, service URLs, SPA fallback, logs and environment parity.

Suggested five-session plan

- Session 1: Diagram the frontend, API, database and Flask service. Trace login end-to-end.
- Session 2: Read User model and auth middleware. Explain hashing, JWT verification and OAuth callback.
- Session 3: Trace alert and nearby providers; explain normalization, caching and failure handling.
- Session 4: Study React routing, the Phaser drill and the Gemini chatbot request/response.
- Session 5: Practice the mock interview and propose security, reliability and testing improvements.

Repository exercise

Pick one endpoint and write down: route path, method, required input, validation, external/database calls, response shape, errors, and which frontend component consumes it. Repeat for auth, alerts, nearby services, and chat.

PHASE 6

System design and scalability

Discuss how to evolve the current multi-service prototype without losing clear ownership.

1. Current request paths

Web: Browser - Vite during development or Vercel static hosting - React Router - Express API - MongoDB Atlas / external providers.

Chat: Browser - React chatbot - Flask /chat - Gemini generateContent - Flask JSON response - React UI.

Map: Browser geolocation - Express /api/nearby - Geoapify Places - normalized locations - list + Leaflet markers.

Alerts: React ticker - Express /api/alerts - NDMA SACHET; client utilities also parse configured USGS/GDACS feeds.

2. What scales and what does not

Current mechanism	Scaling property	Next step
In-memory nearby cache	Per Node process; resets on restart; not shared across replicas.	Use Redis or a managed cache when multiple API instances need shared entries.
MongoDB Atlas	Managed database can scale independently from web workers.	Add indexes, connection limits, backups and least-privilege DB users.
Vercel static frontend	Static assets can be served independently from API compute.	Use immutable hashed assets and monitor rewrite/API availability.
Express APIs	Can run multiple instances behind a load balancer.	Make handlers stateless; externalize sessions/cache and centralize logs.
Flask chatbot	Independent service can scale according to chat traffic.	Add request limits, provider timeouts, auth, queueing and quota controls.
External data providers	Availability/latency/quota are outside the app.	Bound timeouts; cache; add fallbacks and stale-data timestamps.

3. Reliability patterns to explain

- Timeouts: avoid tying up server workers indefinitely while waiting on external APIs.
- Caching: reduce repeat calls and absorb short provider outages; always track freshness.
- Request coalescing: identical in-flight queries share one upstream call.
- Graceful degradation: a failed chatbot or POI provider should not make the rest of the learning platform unusable.
- Observability: record request IDs, status, latency and provider error codes, but redact secrets and personal data.

4. Deployment boundaries

The Vite app is built to static assets; the web host must route frontend paths to index.html and proxy /api and /uploads to the Express service. The Express service needs a correct FRONTEND_URL, Mongo URI, JWT/session settings, OAuth credentials and Geoapify key. The Flask service needs its Gemini key and a stable URL supplied to the frontend build. Local development must configure the Vite proxy and Flask URL separately.

PHASE 7

Security and failure analysis

Know the current trust boundaries and be ready to discuss concrete remediation.

Area	Current concern to verify / improve	Safer direction
JWT in browser	The UI stores user data/token in localStorage; OAuth callback puts token in a redirect URL. Tokens can be exposed through script injection, browser history, logs or referrers.	Use secure, HttpOnly, SameSite cookies or a one-time code exchange; add CSRF protections if cookie auth is used.
Token refresh	The refresh route decodes an expired token payload before reissuing after account lookup. Decoding is not signature verification.	Use short-lived access tokens plus a separately signed, revocable refresh token; verify claims and rotate refresh tokens.
OAuth identity	Callback URL and credentials are environment-specific; default role currently becomes student.	Validate verified email, handle account linking explicitly, protect state/PKCE as appropriate, and constrain callback origins.
Passwords and logs	Some auth handlers log request fields or user identifiers. Passwords and tokens must never appear in logs.	Use structured redacted logs; hash passwords with strong library defaults; rate-limit login and registration.
Profile uploads	Multer checks filename extension and MIME string, but has no visible size limit or content inspection.	Set size limits, verify file signatures, use generated names, store outside web root/object storage, and authorize access.
CORS and sessions	CORS allows configured origins; Express sessions use a fallback secret in code.	Require production secrets, secure cookies, appropriate sameSite, and narrow explicit origins.
Flask/Gemini	Flask enables CORS broadly and debug mode is enabled in the direct-run path; provider request lacks an explicit timeout.	Restrict origins, disable debug in deployment, set request timeouts, rate limits and input-size bounds.
Geoapify key	Provider key is required for POI search and must stay server-side.	Use a server secret manager, restrict key use carefully, rotate exposed keys, and never log the key.
External feed content	Alert/provider content is untrusted and may be malformed or delayed.	Validate expected schemas, escape rendered text, retain source/time metadata and show degraded states.

Failure modes and user-visible behavior

Failure	Expected handling to aim for
MongoDB unavailable	Health endpoint reports degraded status; auth/profile operations return clear 503/5xx without pretending to save.
OAuth misconfigured	Google sign-in returns a clear setup message; password auth remains available.
Geoapify invalid key/quota/timeout	Return sanitized actionable reason; use valid bounded cache when available; log provider status without credentials.
NDMA/USGS/GDACS unavailable	Show last successful alert with timestamp or clearly state feed unavailable; do not imply there are no alerts.
Gemini blocked/error/slow	Return controlled message and retry guidance; do not return provider stack traces or secrets.
Geolocation denied	Explain the fallback location, allow the user to choose a location, and label the map center accurately.

PHASE 8

Mock interview

Use these sample answers as a starting point. Keep the discussion grounded in what is in the repository.

Interviewer: Give me the overview in about one minute.

Candidate: Ready Scout is a disaster-preparedness learning platform. Its main interface is a React app built with Vite, with an Express API and MongoDB Atlas for accounts. It includes preparedness content, alerts, an emergency-services map, a Phaser earthquake drill, and a separate quiz. The chatbot is a Flask service that calls Gemini with a disaster-education prompt. The system is a learning prototype, so I distinguish public alert/provider data from any claim of guaranteed emergency coverage.

Interviewer: Walk me through login.

Candidate: The React login form posts credentials to Express. Express looks up the user in MongoDB through Mongoose, compares the submitted password with the bcrypt hash, signs a JWT and returns the user profile plus token. The frontend stores auth state and includes the token in Bearer authorization for protected calls. Google login uses Passport OAuth and then creates the application JWT at the callback. A security improvement I would prioritize is moving tokens out of localStorage and avoiding tokens in redirect query strings.

Interviewer: Why use a separate Flask service for the chatbot?

Candidate: The repository already has a Python chatbot integration and Flask keeps that provider call isolated from the Node API. The React client sends a message to Flask; Flask adds the disaster-education instruction and calls Gemini. It is easy to reason about, but it adds another deployment and secret. I would add authentication, explicit network timeouts, request limits, rate limiting, error redaction and service health checks.

Interviewer: The nearby map shows no places. What do you inspect?

Candidate: I trace the browser request to /api/nearby, inspect the backend response status and sanitized error, then verify the backend Geoapify key is present and valid, key restrictions, provider quota, categories, and timeout. I confirm the deployed service has the same current code and environment as local. I also check whether cached results are available and whether the selected location is correct. I never expose the API key in frontend variables or logs.

Interviewer: How would you make alerts more dependable?

Candidate: Add a backend aggregation layer for all sources, normalize timestamps and severity consistently, cache last-known-good results with a visible updated-at value, and use timeouts and provider-level error metrics. The UI should distinguish “no active alerts” from “feed unavailable.” For safety-critical use, it would need authoritative source agreements and operational validation.

Interviewer: What would you build next?

Candidate: First I would improve security and testability: validate environment on startup, fix token delivery/refresh, add API schema validation and integration tests, and set upload limits. Then I would improve provider reliability through shared caching, observability and explicit stale-data UX. I would only expand claims about fire/flood workflows after implementing and validating them.

PHASE 9

Final interview cheat sheet

Review these points shortly before an interview or project presentation.

30-second project pitch

“Ready Scout is a disaster-preparedness learning platform built with React and Vite, Node.js/Express, and MongoDB Atlas. It combines learning modules, an earthquake simulation, emergency alerts and a nearby-services map. A separate Flask microservice connects the chatbot to Gemini. I designed the system as an educational prototype and am explicit about external-data, provider-key and emergency-reliability limitations.”

Architecture in one line

React/Vite + React Router -> Express REST API -> MongoDB Atlas; separate Flask -> Gemini chat service; external sources for alerts and nearby POIs; Leaflet map and Phaser game in browser.

Stack

Concern	Technology
Frontend	React, Vite, React Router, Tailwind CSS
API/runtime	Node.js, Express.js
Database/ODM	MongoDB Atlas, Mongoose
Auth	JWT, bcryptjs, Passport.js, Google OAuth 2.0
AI	Flask, Python requests, Gemini API
Map/game	Leaflet.js, OpenStreetMap tiles, Geoapify Places, Phaser 3
Deployment	Vercel frontend, Render backend/services (per current configuration)

Important distinctions

- Vite is the development/build tool; React is the UI library; React Router handles client-side routes.
- Node.js is the runtime; Express.js is the HTTP framework.
- MongoDB Atlas is the managed database service; Mongoose is the Node ODM.
- Leaflet renders maps; Geoapify returns nearby places; OpenStreetMap is an underlying data/tile source.
- Google OAuth authenticates with Google; the app then issues its own JWT.
- A live feed or nearby dataset can be unavailable. “No data” and “no active events” are different states.
- The repository contains fire/flood content pages, but describe them as scaffolded if their interactive workflows are incomplete.

Claims to avoid

- Do not call the system a verified emergency response or dispatch platform.
- Do not claim Geoapify is operational unless the real deployed key has been tested successfully.
- Do not claim the quiz and Phaser drill are one implementation.
- Do not claim secure session handling without acknowledging current browser token storage and OAuth redirect design.

- Do not claim every nearby facility is complete or guaranteed; coverage depends on third-party map data.

Environment variables (names only)

Express: MONGO_URI, JWT_SECRET, SESSION_SECRET, FRONTEND_URL, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GEOAPIFY_API_KEY. Frontend: VITE_FLASK_URL and feed URLs such as VITE_USGS_FEED_URL and VITE_GDACS_FLOOD_RSS. Flask: GEMINI_API_KEY. Keep secret values in local ignored env files or deployment secret settings; do not put them in the frontend bundle.

Local service ports

Service	Typical local URL	Command context
Vite web app	http://localhost:5173	cd frontend; npm run dev
Express API	http://localhost:5000	cd backend; npm run dev
Flask chatbot	http://localhost:5001	cd flask; python app.py
Standalone quiz	Vite-selected port	cd earthquake-quiz; npm run dev

Code landmarks

Frontend routes: frontend/src/App.jsx. Express setup: backend/index.js. User schema: backend/models/User.js. JWT protection: backend/middleware/authMiddleware.js. Auth routes: backend/routes/authRoutes.js. Alerts: backend/routes/alertRoutes.js and frontend/src/lib. Nearby search: backend/routes/nearbyRoutes.js and frontend/src/app/dashboard/student/nearby/page.jsx. Chatbot: flask/app.py and frontend/src/app/dashboard/chatbot/page.jsx. Drill: frontend/public/earthquake_drill/game.js.

Prepared from the Ready Scout repository and current deployment notes. Recheck provider configuration, API behavior, OAuth settings and deployment URLs before interviews, demos or public claims.